



Building blocks

Building blocks, also known as snippets, are how users design and layout pages. They are important XML elements of your design.

The building blocks are classified into two types:

1. **Structure blocks:** visually used as “whole rows” and distributed into multiples categories (**Intro, Columns, Content, Images, People**, etc)
2. **Inner Content blocks:** used inside other building blocks

At the end of this chapter, you will be able to **create custom snippets** and to add them into a dedicated category.

File structure

The view templates live under `views/snippets/` :

```
views
├── snippets
│   ├── snippets.xml
│   └── s_snippet_name.xml
```

Client-side assets live under `static/src/` and are grouped by purpose:

```
static
├── src
│   ├── snippets
│   │   └── s_snippet_name
│   │       ├── snippet_name.js
│   │       ├── snippet_name.edit.js
│   │       ├── 000.scss
│   │       └── 000.xml
│   └── website_builder
│       ├── snippet_name_option.js
│       ├── snippet_name_option.xml
│       └── snippet_name_option_plugin.js
```

Tip

JS Files under `/snippets` are interactions: [example](#) with the `s_floating_blocks` snippet and the base [Interaction](#) class.

Demo page

Demo data have to be installed to access this page:

```
https://your-database.com/website/demo/snippets
```

Layout

Snippets are editable by the user using the Website Builder. Some Bootstrap classes are important as **they trigger some Website Builder options**.

Wrapper

The standard main container of any snippet is a `section` . Any section element can be edited like a block of content that you can move or duplicate.

```
<section class="s_snippet_name" data-name="..." data-snippet="...">
  <!-- Content -->
</section>
```

For inner content snippets, any other HTML tag can be used.

```
<div class="s_snippet_name" data-name="..." data-snippet="...">
  <!-- Content -->
</div>
```

Attribute

Description

class

Unique class name for this snippet

tail back to *BLOCK*.

data-snippet	Used by the system to identify the snippet
---------------------	--

The system automatically adds the `data-name` and `data-snippet` attributes during the drag and drop based on the template's name.

Warning

Those attributes should be specifically added when a snippet is declared on a theme page.

Warning

Avoid adding a `section` tag inside another `section` tag: this will trigger twice the Website Builder's options. You can use inner content snippets instead.

Tip

To write the content of a static page with standard snippets, there are two possible approaches:

- **Pre-build the custom static pages with the Website Builder: Drag & drop snippets, then**
copy/paste the code into your file and clean it up.

OR

- **Code everything directly: But beware of compatibility with the Website Builder.** It requires certain classes, names, id, data, etc. to work properly. A good practice is to seek out the snippets code created in standard code in the Odoo source files. Pay attention that the Website Builder sometimes adds classes to the snippets after dropped in the page.

Elements

There is a list of "features" we can enable/disable by using specific CSS classes.

```
.row > .col-lg-*
```

Add padding on columns and `<section>` .

```
class="pt80 pb80"
```

Note

`pb*` and `pt*` are the Odoo classes used to control the handlers. Their values are increased by **multiples of 8**, till a **max of 256** (0, 8, 16, 24, 32, 40, 48, ...).

Enable the columns selector.

```
<div class="container s_allow_columns">
```

Disable the columns amount option.

```
<div class="row s_nb_column_fixed">
```

Disable the size option for all child columns.

```
<div class="row s_col_no_resize">
```

Disable the size option for one specific column.

```
<div class="col-lg-* s_col_no_resize">
```

Colors

Add a background based on the color palette for columns and `<section>` .

Disable the background color option for all columns.

```
<div class="row s_col_no_bgcolor">
```

Disable the background color option of one specific column.

```
<div class="col-lg-* s_col_no_bgcolor">
```

Add a black color filter with an opacity of 50%.

```
<section>
  <div class="o_we_bg_filter bg-black-50" />
  <div class="container">
    <!-- Content -->
  </div>
</section>
```

Add a white color filter with an opacity of 85%.

```
<section>
  <div class="o_we_bg_filter bg-white-85" />
  <div class="container">
    <!-- Content -->
  </div>
</section>
```

Add a custom color filter.

```
<section>
  <div class="o_we_bg_filter" style="background-color: rgba(39, 110, 114, 0.54) !important" />
  <div class="container">
    <!-- Content -->
  </div>
</section>
```

```
<div class="o_we_bg_filter" style="background-image: linear-gradient(135deg, rgb(
<div class="container">
  <!-- Content -->
</div>
</section>
```

Features

Non-editable areas

Make an element not editable.

```
<div class="o_not_editable">
```

Make an element not removable.

```
<div class="oe_unremovable">
```

Backgrounds

Add a background image and have it centered.

```
<div class="oe_img_bg o_bg_img_center" style="background-image: url('...')">
```

Add parallax effect.

```
<section
  class="parallax s_parallax_is_fixed s_parallax_no_overflow_hidden"
  data-scroll-background-ratio="1">
  <span
    class="s_parallax_bg oe_img_bg o_bg_img_center"
    style="background-image: url('...'); background-position: 50% 75%;" />
  <div class="container">
    <!-- Content -->
```

Note

A video background can be set on a section. Refer to the [Media](#) chapter of this documentation.

Text highlights

Text highlights are SVG files that can be incorporated onto specific words or phrases to emphasize them. Text highlights offer customizable options for colors and thickness.

Title with highlighted text

```
<h2>
  Title with
  <span
    class="o_text_highlight o_text_highlight_fill"
    style="--text-highlight-width: 10px !important; --text-highlight-color: #FFFF
    <span class="o_text_highlight_item">
      highlighted text
      <svg fill="none" class="o_text_highlight_svg o_content_no_merge position-a
        <!-- SVG path -->
      </svg>
    </span>
  </span>
</h2>
```

CSS custom property	Description
--text-highlight-width	Thickness of the SVG borders
--text-highlight-color	Color of the SVG object

Use

Enable the Grid Layout by adding the `o_grid_mode` CSS class on the `row`. The number of rows in your grid is defined by the `data-row-count` attribute. The grid always contains 12 columns. The grid gap, specified in the `style` attribute, determines the gaps (or gutters) between rows and columns.

```
<div class="row o_grid_mode" data-row-count="13" style="gap: 20px 10px">
  <!-- Content -->
</div>
```

Items in a grid

Add items in the grid with the `o_grid_item` class. If the grid item contains an image, use the `o_grid_item_image` class.

```
<div class="row o_grid_mode" data-row-count="13">
  <div class="o_grid_item g-height-* g-col-lg-*" style="grid-area: 2 / 1 / 7 / 8; ">
    <!-- Content -->
  </div>
  <div class="o_grid_item o_grid_item_image g-height-* g-col-lg-*" style="grid-area: 2 / 1 / 7 / 8; ">
    
  </div>
</div>
```

The dimensions and position of a grid item are defined by the `grid-area` that can be explicitly set in the `style` attribute along with the `z-index`.

The `g-height-*` and `g-col-lg-*` classes are generated by the Website Builder for editing purposes.

Grid item padding

```
<div class="row o_grid_mode" data-row-count="13" style="gap: 20px 10px;">
  <div class="o_grid_item g-height-* g-col-lg-*" style="--grid-item-padding-y: 20px;">
    <!-- Content -->
  </div>
</div>
```

CSS custom property	Description
<code>--grid-item-padding-y</code>	Vertical paddings (Y axis)
<code>--grid-item-padding-x</code>	Horizontal paddings (X axis)

Compatibility system

When a snippet has a `data-vcss` , `data-vjs` and/or `data-vxml` attribute, it means it is an updated version, not the original one.

```
<section class="s_snippet_name" data-vcss="001" data-vxml="001" data-vjs="001">
  <!-- Content -->
</section>
```

These data attributes indicate to the system which file version to load for that snippet (e.g., `001.js` , `002.scss`).

Custom snippet

Some more specific needs will require the creation of custom snippets. Here is how to create a custom snippet/

Template

Create first the snippet template. Then, add it to the list and make it available via the Website Builder.

1. Declaration

First, create the template of the custom snippet:

```
/website_airproof/views/snippets/s_airproof_snippet.xml
```

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
```

```
</template>
```

```
</odoo>
```

Warning

`data-name` and `data-snippet` attributes have to be specified when a snippet is declared on a theme page. Otherwise, the snippet won't be recognised by the Website Builder and issues might occur whenever a database upgrade is done. Additionally, remember that the name attribute is shown as the name of your custom snippet in the **Blocks** section of the options panel.

Tip

- Use Bootstrap native classes as much as possible.
- Prefix all your custom classes.
- Use underscore lowercase notation to name classes, e.g., `.x_nav`, `.x_nav_item`.
- Avoid using ID attribute within your `section` as several instances of a snippet may appear throughout the page (An ID attribute has to be unique on a page).

Add the custom snippet to the list of standard snippets, so the user can drag and drop it on the page, directly from the edit panel.

2. Group creation

Add a group at the top of the list (feel free to put it where needed in this list).

```
/website_airproof/views/snippets/snippets.xml
```

```
<template id="snippets" inherit_id="website.snippets" name="
  <!-- Create the group -->
  <xpath expr="//snippets[@id='snippet_groups']/*[1]" posi
    <t snippet-group="airproof"
      t-snippet="website.s_snippet_group"
      string="Airproof"
      t-thumbnail="/website_airproof/static/src/img/wbui
```

Attribute	Description
snippet-group	ID of the group
t-snippet	Inherited template ID
string	Group name displayed to the users
t-thumbnail	The path to the thumbnail of the group

1. Snippet addition

Then add the custom snippet into the `<snippets id="snippet_structure">` which contains all existing ones on the same level. The Website Builder will split them automatically into categories by reading the `group` attribute on the `<t t-snippet="">`

/website_airproof/views/snippets/snippets.xml

```
<template id="snippets" inherit_id="website.snippets" name="
  <!-- Create the group -->
  <xpath expr="//snippets[@id='snippet_groups']/*[1]" posi
    <t snippet-group="airproof"
      t-snippet="website.s_snippet_group"
      string="Airproof" t-thumbnail="/website_airproof/s
    </xpath>

  <!-- Add the custom snippet to the group -->
  <xpath expr="//snippets[@id='snippet_structure']/*[1]" po
    <t t-snippet="website_airproof.s_airproof_snippet" st
      <keywords>Snippet</keywords>
    </t>
  </xpath>
</template>
```

Attribute	Description
t-snippet	The snippet template to use

<keywords> Keywords entered by the user in the search field in the Snippets panel

Inner content snippet

To make a custom snippet appear in the **Inner content** list, add it to `snippet_content` instead:

/website_airproof/views/snippets/snippets.xml

```
<template id="snippets" inherit_id="website.snippets" name="
  <!-- Add the custom snippet to the group -->
  <xpath expr="//snippets[@id='snippet_content']/*[1]" pos:
    <t t-snippet="website_airproof.s_airproof_snippet"
      string="Custom snippet"
      t-thumbnail="/website_airproof/static/src/img/wbui
  </xpath>
</template>
```

Inner content selectors

To make a custom snippet “inner content” (droppable in another building block), add its selector to the `so_content_addition_selector` resource, which contains all selectors for existing inner content building blocks.

/website_airproof/static/src/website_builder/airproof_snippet_option_plugin.js

```
export class AirproofSnippetOptionPlugin extends Plugin {
  static id = "airproofSnippetOption";
  resources = {
    so_content_addition_selector: [".s_airproof_snippet"],
  };
}
```

Important

- Don't forget to add a `t-thumbnail` and remove the `group` attribute as this kind of building blocks is directly available in the right options panel of the Website Builder.

Options allow users to edit a snippet's appearance and behavior using the Website Builder. Options are OWL components defined in JavaScript and rendered with XML templates. Each option is registered through a builder plugin and loaded in the editor assets.

Note

Add the JavaScript and XML files to the `website.website_builder_assets` bundle in your module.

See also

- [HTML Builder options](#)
- [Website Builder options](#)
- [Alert option example](#)

Component and template

Define the option component and register it in a builder plugin.

```
/website_airproof/static/src/website_builder/airproof_snippet_option_plugin.js
```

```
import { BaseOptionComponent } from "@html_builder/core/uti
import { Plugin } from "@html_editor/plugin";
import { registry } from "@web/core/registry";

export class AirproofSnippetOption extends BaseOptionComponent {
    static template = "website_airproof.AirproofSnippetOption
    static selector = ".s_airproof_snippet";
    static applyTo = ":scope > .row";
}

export class AirproofSnippetOptionPlugin extends Plugin {
    static id = "airproofSnippetOption";
    resources = {
        builder_options: [AirproofSnippetOption],
    };
}
```

Then define the XML template rendered in the options panel.

/website_airproof/static/src/website_builder/airproof_snippet_option.xml

```
<templates xml:space="preserve">
  <t t-name="website_airproof.AirproofSnippetOption">
    <BuilderRow label.translate="Layout">
      <BuilderSelect>
        <BuilderSelectItem classAction="">Default</Bu
        <BuilderSelectItem classAction="'s_airproof_snip
        <BuilderSelectItem classAction="'s_airproof_snip
        <BuilderSelectItem classAction="'s_airproof_snip
      </BuilderSelect>
    </BuilderRow>
    <BuilderRow label.translate="Space">
      <BuilderButtonGroup>
        <BuilderButton classAction="'mt-0'">1</BuilderBu
        <BuilderButton classAction="'mt-3'">2</BuilderBu
        <BuilderButton classAction="'mt-5'">3</BuilderBu
      </BuilderButtonGroup>
    </BuilderRow>
  </t>
</templates>
```

Option logic

Define option behavior in JavaScript on the `BaseOptionComponent` class and register any `BuilderAction` used by the template in the builder plugin resources.

Plugin API

Builder plugins register option components, actions, and editor hooks through the `resources` object. They are added to the `website-plugins` registry category to be accessible by the editor. Plugin instances are created when the editor starts and destroyed when the editor closes, they live for the whole editor session.

static shared	Expose shared methods to other plugins.
----------------------	---

setup()	Initialize plugin state.
----------------	--------------------------

Resources

Purpose

builder_options	Register <code>BaseOptionComponent</code> classes for the options panel.
builder_actions	Register <code>BuilderAction</code> classes referenced by option templates.
dropzone_selector	Define snippet drop rules (<code>selector</code> , <code>dropIn</code> , <code>dropNear</code> , <code>exclude</code> , <code>excludeNearParent</code>).
so_content_addition_selector	Extend the list of inner content selectors.
Handlers	Editor lifecycle hooks registered on the plugin.
on_snippet_dropped_handlers	Called after a snippet is dropped and inserted in the DOM.
on_cloned_handlers	Called after an element is cloned and inserted in the DOM.
on_will_remove_handlers	Called just before an element is removed.
on_removed_handlers	Called after an element has been removed.
clean_for_save_handlers	Called on a cleaned clone of the DOM before saving.
before_save_handlers	Called at the beginning of the save process.
after_save_handlers	Called after the save process completes.

See also

[Full resources list](#) 

BaseOptionComponent API

short lifecycle.

Property	Description
static template	OWL template name rendered in the options panel.
static selector	CSS selector that binds the option to elements.
static exclude	CSS selector of elements to exclude from the match.
static applyTo	CSS selector targeting a child element of the matched snippet.
static components	Child builder components used in the template.
setup()	Initialize component state.

Binding

selector

Option components declare a selector property that controls when they appear.

`selector` binds the option component to elements matching a CSS selector (class, ID, etc). The option appears when a matching element is selected.

```
export class AirproofSnippetOption extends BaseOptionComponent {
    static selector = "section, h1, .custom_class, #custom_id";
}
```

It can be used in combination with `exclude` or `applyTo`.

applyTo

`applyTo` allows the option to apply to a child element of the matched selector.

```
export class AirproofSnippetOption extends BaseOptionComponent {
    static selector = ".s_airproof_snippet";
    static applyTo = ".row";
}
```

The option appears if an `<ul>` tag (without `.navbar-nav` class) is selected

```
export class AirproofSnippetOption extends BaseOptionComponent {
    static selector = "ul";
    static exclude = ".navbar-nav";
}
```

dropzone_selector (dropIn)

`dropIn` defines the elements the snippet can be dropped into.

```
export class AirproofSnippetOptionPlugin extends Plugin {
    static id = "airproofSnippetOption";
    resources = {
        dropzone_selector: {
            selector: ".s_airproof_snippet",
            dropIn: ".x_custom_location",
        },
    };
}
```

dropzone_selector (dropNear)

`dropNear` defines the elements the snippet can be dropped next to.

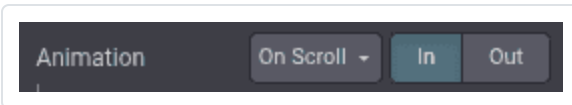
```
export class AirproofSnippetOptionPlugin extends Plugin {
    static id = "airproofSnippetOption";
    resources = {
        dropzone_selector: {
            selector: ".s_airproof_snippet_card",
            dropNear: ".card",
        },
    };
}
```

Layout & fields

Use builder components to compose the options panel UI.

```
<BuilderRow label.translate="My option">
  <BuilderSelect>...</BuilderSelect>
  <BuilderButtonGroup>...</BuilderButtonGroup>
</BuilderRow>
```

The perfect example for this case is the **Animation** row:



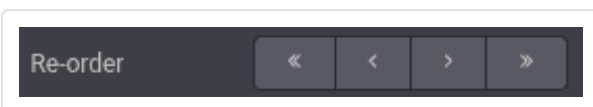
`<BuilderButton>`

This tag is used inside `<BuilderButtonGroup>` or on its own.

```
<BuilderButtonGroup>
  <BuilderButton classAction="'mt-0'">1</BuilderButton>
  <BuilderButton classAction="'mt-3'">2</BuilderButton>
  <BuilderButton classAction="'mt-5'">3</BuilderButton>
</BuilderButtonGroup>
```

Add `classAction` to indicate which class is applied to the targeted element when this choice is selected. Like any XML node, add other attributes to improve the style and/or the user experience.

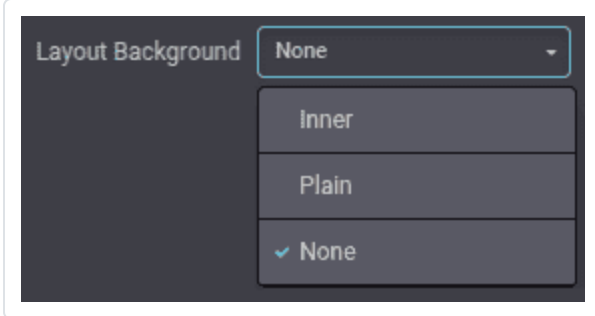
```
<BuilderButton
  icon="'fa-align-right'"
  title.translate="Align right"
  classAction="'ms-auto'" />
```



`<BuilderSelect>`

Formats the option as a dropdown list.

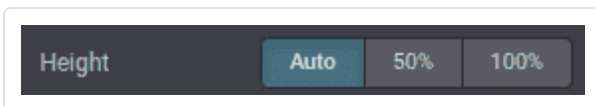
```
</BuilderSelect>
```



```
<BuilderButtonGroup>
```

Formats the option as buttons next to each other and handles selection state.

```
<BuilderButtonGroup>...</BuilderButtonGroup>
```



```
<BuilderCheckbox>
```

Formats the option as a toggle switch.

```
<BuilderCheckbox  
  classAction="'s_airproof_snippet_tooltip'" />
```



```
<BuilderRange>
```

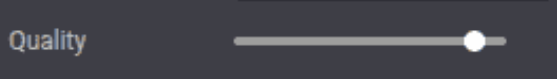
Formats the option as a slider.

`<BuilderRange>` can be used to set a numeric value:

```
max="10"
step="1"
displayRangeValue="true" />
```

It can also be used to map each step to an entry in `actionParam` :

```
<BuilderRange
  action="'setClassRange'"
  actionParam=['o_spc-none', 'o_spc-small', 'o_spc-medium', 'o_spc-big']"
  max="3" />
```



`<BuilderNumberInput>` and `<BuilderTextInput>`

Format the option as a numeric or text field.

`unit`, `saveUnit` and `step` are optional

```
<BuilderNumberInput
  styleAction="'animation-duration'"
  unit="'s'"
  saveUnit="'ms'"
  step="0.1" />
```



`<BuilderNumberInput>` comes with optional props:

Attribute	Description
<code>unit</code>	Shows the expected unit of measure.

is converted and saved.

step	Set the numerical value by which the field can be incremented.
------	--

<BuilderList>

Render and edit a list of items (add, remove, reorder) in the options panel. Useful for prefilled values, custom options, or settings that map to a list of objects.

Simple list with two fields

```
<BuilderList
  action="'setPrefilledOptions'"
  addItemTitle.translate="Add"
  itemShape="{ value: 'number', description: 'text' }"
  default="{ value: '50', description: 'Add a description I
  columnWidth="{ value: 'w-25' }"
  sortable="true" />
```

Prop	Description
itemShape	Field types per column. Supported types: text , number , boolean , exclusive_boolean .
default	Default values for new items. Keys must match itemShape .
records	Optional list of available records (JSON list). When set, the add button becomes a dropdown.
defaultNewValue	Extra values injected when creating a new item (e.g., record ID).
sortable	Allow drag-and-drop reordering (default: true).
hiddenProperties	Hide specific columns by name.
columnWidth	CSS classes applied per column (e.g., w-25).
forbidLastItemRemoval	Prevent removing the last remaining item.

<BuilderColorPicker>

```

action="'selectFilterColor'"
defaultOpacity="50"
enabledTabs=['custom', 'gradient']" />

```

Background



Attribute

Description

enabledTabs

Restrict the available picker tabs (e.g., ['theme', 'gradient', 'custom']).

defaultColor

Default color shown when no selection is applied.

defaultOpacity

Default alpha value for the picker.

Actions

Builder components trigger `BuilderAction` classes. Use shorthand action props or declare custom actions and reference them by `action id` in the template.

Core actions

Core actions are available as shorthand props on builder components.

Action/prop

Description

classAction

Add/remove CSS classes on the target element.

styleAction

Set an inline style property on the target element.

attributeAction

Set or remove a DOM attribute.

dataAttributeAction

Set or remove a dataset key (without the `data-` prefix).

setClassRange

Apply one class from a predefined list based on the selected range value.

action + actionParam

Call a custom action registered in the builder plugin.

Method	Description
prepare	Prepare asynchronous data before a component is used.
load	Load data before <code>apply</code> when actions are previewed.
apply	Apply the action to the targeted element.
clean	Reset the action when a new choice supersedes it.
getValue	Return the current value for input widgets.
isApplied	Return whether the action is active for selectable widgets.
getPriority	Decide which selectable item wins when multiple are valid.

Custom actions

To create custom behaviors, define a `BuilderAction` and register it in your builder plugin.

```
/website_airproof/static/src/website_builder/airproof_snippet_option_plugin.js
```

```
import { BuilderAction } from "@html_builder/core/builder_action";
import { Plugin } from "@html_editor/plugin";
import { registry } from "@web/core/registry";

export class AirproofLayoutAction extends BuilderAction {
    static id = "airproofLayout";
    apply({ editingElement, params: { mainParam } }) {
        editingElement.classList.toggle("s_airproof_snippet_preview");
        editingElement.classList.toggle("s_airproof_snippet_selected");
    }
    isApplied({ editingElement, params: { mainParam } }) {
        return editingElement.classList.contains("s_airproof_snippet_selected");
    }
}

export class AirproofSnippetOptionPlugin extends Plugin {
    static id = "airproofSnippetOption";
    resources = {
```

```
registry.category("website-plugins").add(
    AirproofSnippetOptionPlugin.id,
    AirproofSnippetOptionPlugin
);
```

Finally, the custom action can be called from the XML template.

```
<t t-name="website_airproof.AirproofSnippetOption">
  <BuilderRow label.translate="Layout">
    <BuilderButtonGroup action="'airproofLayout'">
      <BuilderButton actionParam="'portrait'">Portrait</BuilderButton>
      <BuilderButton actionParam="'square'">Square</BuilderButton>
    </BuilderButtonGroup>
  </BuilderRow>
</t>
```

See also

- [Builder actions API](#)

Dynamic Content templates

By default, Dynamic Content blocks have a selection of templates available in the Website Builder. Custom templates can also be added to the list automatically by use the same naming convention for the template id attribute.

Call the template

The selected dynamic snippet replaces the `<div class="dynamic_snippet_template"/>` placeholder by the right template based on the `data-template-key` and the custom CSS class:

```
<section
  data-snippet="s_blog_posts"
  data-name="Blog Posts"
  class="s_blog_post_airproof s_dynamic_snippet_blog_posts s_blog_posts_post_pictu
  data-template-key="website_airproof.dynamic_filter_template_blog_post_airproof"
  data-custom-template-data="{
```

```

        blog_posts_post_new_active:true
    }"
    data-number-of-records="16"
    data-filter-by-blog-id="-1"
    data-filter-id="1"
    data-extra-classes="g-3"
    data-column-classes="col-12 col-sm-6 col-lg-4 col-xl-3">
    <div class="s_dynamic_snippet_container container">
        <div class="row s_nb_column_fixed">
            <section class="s_dynamic_snippet_holder d-none px-4 placeholder-glow o
                <div class="row">
                    <span class="placeholder col-3 rounded" />
                    <span class="placeholder col-2 offset-7 rounded" />
                    <span class="placeholder mt-3 col-6 rounded" />
                </div>
                <div class="row mt-4">
                    <span class="placeholder col-12 rounded" style="height: 250px;" />
                </div>
            </section>
            <section class="s_dynamic_snippet_title oe_unremovable oe_unmovable fle
                <div>
                    <h2 class="h3">Our latest content</h2>
                    <p class="lead">Check out what's new in our company!</p>
                </div>
                <div>
                    <a title="See All" href="/blog">
                        See all<span class="fa fa-long-arrow-right ms-2" />
                    </a>
                </div>
            </section>
            <section class="s_dynamic_snippet_content oe_unremovable oe_unmovable o
                <div class="css_non_editable_mode_hidden">
                    <div class="missing_option_warning alert alert-info fade show d-n
                        Your Dynamic Snippet will be displayed here... This message
                    </div>
                </div>
                <div class="dynamic_snippet_template" />
            </section>
        </div>
    </div>
</section>

```

rendering.

Examples

Blog postsProductsEvents

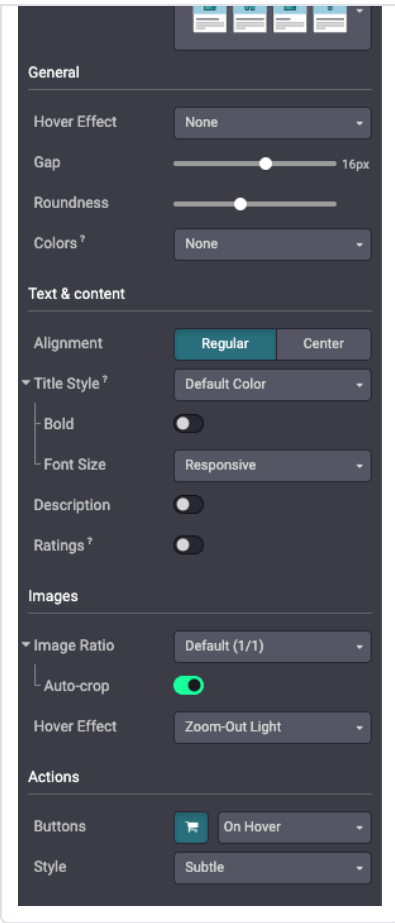
/website_airproof/views/snippets/s_snippet_name.xml

```
<template id="dynamic_filter_template_blog_post_airproof"
  <div t-foreach="records"
    t-as="data"
    class="s_blog_posts_post"
    data-extra-classes="g-3"
    data-column-classes="col-12 col-sm-6 col-lg-4 col-
  <t t-set="record" t-value="data['_record']" />
  <!-- Content -->
  </div>
</template>
```

Attribute	Description
id	The ID of the template. Has to start with dynamic_filter_template_blog_post_
name	Human-readable name of the template
data-extra-classes	CSS classes added on the parent .row of the blog post columns.
data-column-classes	CSS classes added on the parent column of the blog post cards.

Dynamic products

The dynamic products snippet introduces a bunch of micro-styling options under **Cards Design**. To set one of these options, insert the related CSS class into the `section` . Aside from some exceptions that rely on CSS variables, adding the class sets the option to the right value.



Card options

Here is non-exhaustive list of these available options:

General			
Text & Content			
Images			
Actions			
Option	Description	CSS Classes / Values	
Hover Effect	Background color on hover	o_wsale_products_opt_hover_backgr	
	Background Zoom on hover	o_wsale_products_opt_hover_backgr o_wsale_products_opt_hover_backgr	
Gap	Spacing between	For example: style="--o-wsale-products-grid-ga	

	0 to 28px)	
Roundness	Border radius of the product cards	o_wsale_products_opt_rounded_0
		o_wsale_products_opt_rounded_1
		o_wsale_products_opt_rounded_2
		o_wsale_products_opt_rounded_3
		o_wsale_products_opt_rounded_4
		o_wsale_products_opt_rounded_5
Colors	Applies color combinations defined in the Theme tab	.o_wsale_products_opt_cc o_wsale_products_opt_cc1
		o_wsale_products_opt_cc o_wsale_products_opt_cc2
		o_wsale_products_opt_cc o_wsale_products_opt_cc3
		o_wsale_products_opt_cc o_wsale_products_opt_cc4
		o_wsale_products_opt_cc o_wsale_products_opt_cc5

Card presets

Instead of being separately set, a **Preset** can also be selected. This has a layout class (Catalog or List), a specific styling class and all the class options (see above). This dynamic snippet is forced to catalog display (`o_wsale_products_opt_layout_catalog`) and one of the following preset:

Preset	CSS Class
Thumbnails	o_wsale_products_opt_design_thumbs



Cards	<code>o_wsale_products_opt_design_cards</code>
--------------	--

Grid	<code>o_wsale_products_opt_design_grid</code>
-------------	---

Get Help

[Contact Support](#)[Ask the Odoo Community](#)